



Protocol Audit Report

Version 1.0

Suryansh

September 24, 2026

Protocol Audit Report

Suryansh

September 23, 2026

Prepared By

Prepared by: Suryansh Porwal

Lead Auditors:

- Suryansh

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues Found
- Findings
 - High
 - * [H-1] Storing the password on-chain makes it visible to anyone, and no longer private
 - Likelihood and Impact

- * [H-2] PasswordStore::setPassword() has no access controls, meaning a non-owner could change the password
 - Likelihood and Impact
 - Medium
 - Low
 - Informational
- * [I-1] Wrong NatSpec: The PasswordStore::getPassword NatSpec indicates a parameter that doesn't exist, causing the NatSpec to be incorrect
 - Likelihood and Impact
- Gas

Protocol Summary

A smart contract application for storing a password. Users should be able to store a password and then retrieve it later. Others should not be able to access the password.

Disclaimer

The HydrousAudits team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

7d55682ddc4301a7b13ae9413095feffd9924566

Scope

```
./src/  
#--PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

Executive Summary

Add some notes about how the audit went, types of things you found, etc.

We spend X hours with Z auditors using Y tools. etc

Issues Found

Severity	Number of issues found
High	2
Medium	0
Low	0
Informational	1

Findings

High

[H-1] Storing the password on-chain makes it visible to anyone, and no longer private

Description

All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword()` function, which is intended to be only called by the owner of the contract.

We can show one such method of reading any data off chain below.

Impact

Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept (Proof of Code)

The below test case shows how anyone can read data from the private variable in the code.

1. Create a locally running anvil node:

```
make anvil
```

1. Deploy contract to chain:

```
make deploy
```

1. Run the storage tool.

Recommended Mitigation

Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

Likelihood and Impact

- **Impact:** HIGH
- **Likelihood:** HIGH
- **Severity:** HIGH

[H-2] PasswordStore::setPassword() has no access controls, meaning a non-owner could change the password**Description**

The PasswordStore::setPassword() function is set to be an external function, however the NatSpec of the function and overall purpose of the smart contract is that This function allows only the owner to set a new password.

```
function setPassword(string memory newPassword) external {  
    // no access controls applied  
    s_password = newPassword;  
    emit SetNewPassword();  
}
```

Impact

Any unauthorised user can set/change the password of the contract, severely breaking the contract intended functionality.

Proof of Concept

Add the following to the PasswordStore.t.sol test file.

Code

```
function test_non_owner_can_set_password(address randomAddress) public {  
    vm.assume(owner != randomAddress);  
  
    vm.prank(randomAddress);  
  
    string memory expectedPassword = "myPassword";  
  
    passwordStore.setPassword(expectedPassword);  
  
    vm.prank(owner);  
  
    string memory actualPassword = passwordStore.getPassword();  
  
    assertEq(actualPassword, expectedPassword);  
}
```

Recommended Mitigation

Add an access control conditional to the setPassword function.

```
if (msg.sender != s_owner) revert PasswordStore__NotOwner();
```

Likelihood and Impact

- **Impact:** HIGH
- **Likelihood:** HIGH
- **Severity:** HIGH

Medium**Low****Informational**

[I-1] Wrong NatSpec: The PasswordStore::getPassword NatSpec indicates a parameter that doesn't exist, causing the NatSpec to be incorrect

Description

The PasswordStore::getPassword function signature is getPassword while the NatSpec says it should be getPassword(string).

```
/**
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory) {
    ...
}
```

Impact

The NatSpec is incorrect.

Recommended Mitigation

Remove the incorrect NatSpec line.

```
-    * @param newPassword The new password to set.
```

Likelihood and Impact

- **Impact:** None
- **Likelihood:** HIGH
- **Severity:** Informational/Gas/Non-crits

Gas